

	Nicola Beccaceci s360641
Iniziato	3 luglio 2026, 14:01
Stato	Completato
Terminato	3 luglio 2026, 15:06
Tempo impiegato	1 ora 5 min.
Valutazione	2,0 su un massimo di 6,0 (33%)
Riepilogo del tentativo	1

Domanda 1

Completo

Punteggio
ottenuto 2,0
su 6,0

ResourcePool



Molti servizi reali non aprono una nuova connessione ogni volta che ne hanno bisogno: l'apertura è lenta e onerosa. Mantengono invece un insieme fisso di connessioni già pronte in un **pool**; chi deve lavorare ne preleva una, la usa e la restituisce, affinché torni disponibile per qualcun altro. È lo schema dei **connection pool** usati in molti contesti, come i database o i servizi web.

Si scriva in Rust una struttura che implementi il tratto generico `ResourcePool<T: Send>`, che gestisce un insieme fisso di elementi riutilizzabili di tipo `T` e li concede in prestito esclusivo ai thread che ne fanno richiesta. Questi elementi devono essere racchiusi da un tipo che implementi il tratto generico `Resource<T: Send>`, che permette di accedere al valore dell'elemento concesso in prestito e che li restituisce al pool quando il valore esce dallo scope utilizzando il concetto di RAII (tramite il tratto `Drop`).

API richiesta

```
pub trait Resource<T: Send> {  
    fn get(&self) -> &T;  
}  
  
pub fn make_resource_pool<T: Send>(items: Vec<T>) -> impl ResourcePool<T> {  
    ...  
}  
  
pub trait ResourcePool<T: Send> {  
    // Numero totale di elementi gestiti dal pool.  
    fn capacity(&self) -> usize;  
  
    // Preleva un elemento dal pool e lo consegna al chiamante. Se nessun  
    // elemento è disponibile, blocca il chiamante senza consumare cicli di CPU  
    // finché una risorsa non viene rilasciata.  
    fn acquire(&self) -> impl Resource<T>;  
  
    // Variante con attesa limitata: come acquire, ma se non ottiene un  
    // elemento entro timeout rinuncia e restituisce None. L'attesa non deve  
    // consumare CPU.  
    fn acquire_timeout(&self, timeout: Duration) -> Option<impl Resource<T>>;  
}
```

Requisiti

- La struttura deve essere thread-safe e condivisibile tra più thread.
- Ogni elemento deve essere concesso ad al più un thread alla volta: quando l'oggetto che implementa il tratto `Resource<T>` viene distrutto, il valore di tipo `T` (con il proprio stato) presente al suo interno torna nel pool, diventando disponibile per un'ulteriore richiesta.
- Nessuna attesa attiva (niente busy-waiting).

I test in fondo al file `src/lib.rs` devono passare senza modifiche (`cargo test`).

Se il codice consegnato non compila, non verrà valutato.

*La risposta a questa domanda è stata inserita tramite un'istanza Crownlabs ed è visibile sul Portale della Didattica sulla sezione **Consegna Elaborati** di questo insegnamento.*

Commento:

Il codice compila ma passano 6 test su 12 (5 falliscono e 1 resta bloccato in modo indefinito).

Manca l'implementazione del metodo `get()` per il tipo `Element<T>`.

Manca l'implementazione del tratto `Drop` per il tipo `Element<T>`.

La struttura dati che hai scelto per il tipo `Element<T>` non ti consente di risolvere l'esercizio.

La struttura dei metodi della struct `Manager` è tutta da rivedere.

Una possibile soluzione vagamente basata sul tuo codice è:

```
pub mod pool {
    use std::{sync::{Arc, Condvar, Mutex}, time::Duration};

    pub trait Resource<T: Send> {
        /// Accesso condiviso (`&T`) all'elemento: per modificarne lo stato serve
        /// la mutabilità interna (es. tipi atomici o `Mutex`).
        fn get(&self) -> &T;
    }

    pub struct Element<T: Send> {
        element: Option<T>,
        state: Arc<(Mutex<SharedState<T>>, Condvar)>
    }

    impl<T: Send> Element<T> {
        pub fn with_element_and_state (_item: T, state: Arc<(Mutex<SharedState<T>>, Condvar)>) -> Self {
            Self {
                element: Some(_item),
                state
            }
        }
    }

    impl<T: Send> Resource<T> for Element<T> {
        fn get(&self) -> &T {
            // let value = &self.element.unwrap();
            // value
            // non sono riuscito a finirlo per mancanza di tempo
            self.element.as_ref().unwrap()
        }
    }

    impl<T: Send> Drop for Element<T> {
```

```

fn drop(&mut self) {
    let item = self.element.take().unwrap();
    self.state.0.lock().unwrap().resources.push(item);
    self.state.1.notify_all();
}
}

pub struct SharedState<T: Send> {
    resources: Vec<T>,
}

impl<T: Send> SharedState<T> {

    pub fn with_elements (_items: Vec<T>) -> Self {
        Self {
            resources: _items,
        }
    }
}

pub struct Manager<T: Send> {
    state: Arc<(Mutex<SharedState<T>>, Condvar)>,
    capacity: usize,
}

impl<T: Send> Manager<T> {

    pub fn with_elements (_items: Vec<T>) -> Self {
        let capacity = _items.len();
        Self {
            state: Arc::new((Mutex::new(SharedState::with_elements(_items)), Condvar::new())),
            capacity
        }
    }
}

impl<T: Send> ResourcePool<T> for Manager<T> {

    fn capacity(&self) -> usize {
        self.capacity
    }

    fn acquire(&self) -> impl Resource<T> {
        let (lock, cvar) = &*self.state;
        let mut guard = lock.lock().unwrap();
        guard = self.state.1.wait_while(guard, |g|g.resources.is_empty()).unwrap();
    }
}

```

```

        let item = guard.resources.pop().unwrap();
        Element::with_element_and_state(item, self.state.clone())
    }

    fn acquire_timeout(&self, timeout: Duration) -> Option<impl Resource<T>> {
        let (lock, cvar) = &*self.state;
        let guard = lock.lock().unwrap();
        let (mut guard, timedOut) = cvar.wait_timeout_while(guard, timeout,
|g|g.resources.is_empty()).unwrap();
        if timedOut.timed_out() { return None }
        let item = guard.resources.pop().unwrap();
        Some(Element::with_element_and_state(item, self.state.clone()))
    }
}

pub trait ResourcePool<T: Send> {
    /// Numero totale di elementi gestiti dal pool.
    fn capacity(&self) -> usize;
    /// Preleva un elemento dal pool, attendendo senza consumare CPU se
    /// nessuno e' disponibile.
    fn acquire(&self) -> impl Resource<T>;
    /// Variante con attesa limitata: come [ `acquire` ](Self::acquire), ma se
    /// non ottiene un elemento entro `timeout` restituisce `None`.
    fn acquire_timeout(&self, timeout: Duration) -> Option<impl Resource<T>>;
}

pub fn make_resource_pool<T: Send>(_items: Vec<T>) -> impl ResourcePool<T> {
    Manager::with_elements(_items)
}
}

```